BACKEND HANDOVER

Backend task list

What the backend still needs, in priority order, including every decision answered in the Gloxad LMS Backend Technical Specification (v1.1).

Gloxad Academy | Prepared 11 November 2026 | Repository: gloxard-backend (Express 5, MongoDB)

HOW TO USE THIS LIST

1. Summary

The backend already runs authentication, courses, enrolment and payment, the learning engine, messaging, and the trimester LMS API. This list has four parts, in the order the work should be done:

- **A. Written already, needs review and release.** Code I added to the backend repository that has not been committed or deployed.
- **B. Production steps.** What has to be run or configured before any of it is live.
- **C. From the specification (v1.1, 6 November 2026).** The approved decisions and the backend work each one requires, by the spec's own phases.
- **D. Earlier gaps** from the web app that are not in the specification.

The specification says engineering should first compare its requirements with the existing codebase and record reusable services, conflicts and gaps. Part C does that for each area, so it can be copied into the team's own plan.

A. Written already: review, commit, release

All of this works and was tested against a local database. The older learning-engine test still passes (19 of 19).

Item	What it does and where it is	Status
Module-by-module free courses	Lessons, then quiz, then written task, then the next module. Enforced on the server (403 MODULE_LOCKED, 409 LESSONS_INCOMPLETE, 409 QUIZ_NOT_PASSED). Files: src/utils/moduleGate.js, src/controllers/moduleLearning.controller.js, changes in learning.controller.js and learning.routes.js.	Uncommitted
New endpoints	GET /api/learning/:courseId/modules; GET and POST /api/learning/:courseId/modules/:moduleId/task (min words, optional https link, every submission kept, latest counts, duplicate send is not a new attempt).	Uncommitted
Data model	Course.sequentialModules flag and a task on each module; Enrollment.taskSubmissions. The flag is set only by the catalogue import, so tutor courses keep optional quizzes.	Uncommitted
Generated quizzes and tasks	src/utils/moduleAssessments.js builds up to five questions and a task for every module of the 74 free courses from the catalogue data. Deterministic; quiz ids survive re-imports. Replace with tutor-written questions over time.	Uncommitted
Quiz answer key	The correct option is returned in the quiz result breakdown only after an attempt, never with the course.	Uncommitted
Tests	scripts/test-module-gating.js (27 checks, repeatable). scripts/test-unit12.js was updated for the answer key change.	Uncommitted
Progress rule	On sequential courses progress counts lessons + quizzes + tasks. A finished course stays finished if quizzes are added later.	Uncommitted
Scholarships and public trimester pages	Scholarship and application models, public lists, apply, admin create/publish/review, trimester enrol and status update.	Already committed

B. Production steps

- Commit and merge the work in part A, then deploy the backend. The Render service has not deployed since 27 October.
- Run **npm run catalog:import** on the production database. This adds the quizzes and tasks to the 74 free courses and sets sequentialModules.
- Decide how to treat learners already part-way through a free course: they will now need the quiz and task of earlier modules. Either accept it (nothing breaks, finished courses keep their certificate) or add a one-off migration that marks earlier modules complete.
- Confirm environment variables on Render (MongoDB, JWT secret, SES, Paystack, S3, CORS origin for the new web domain). Rotate the seed password: it appears in a public repository.

- Keep the catalogue JSON in step: the web app's src/constants/catalog/gloxadCourses.json and the backend's src/data/gloxadCourses2026.json must match (the import header says so).

C. From the specification

The specification (Gloxad LMS Backend Technical Specification, v1.1, 6 November 2026) records seven answered questions. Rules: **must** is required, **should** is strong advice, **may** is optional. Example API paths are illustrative unless marked approved. The tables give each decision and the backend work it needs.

Principles that apply to everything

- Configurable by data, not hard-coded: categories, providers, programme structure, assessment types, weights, deadlines.
- The backend is the source of truth for curriculum order, progress, grades, recording state and workflow state; the frontend only displays it.
- Role checks on every protected resource (learner, tutor or instructor, administrator).
- Audit history for grade changes, approvals, policy changes, formal submissions and sensitive access.
- Keep these separate: watching a recording is not live attendance; saving a draft is not submission; a GitHub push is not submission; a browser test score is not a verified grade; a tutor's final-project recommendation is not official until the administrator approves it.

Phase 1: core academic foundation

Question and decision	Backend work	Priority
Q1. 18 primary categories (Software Development, AI, Data Science, Cloud, Cybersecurity, Product and UI/UX Design, Digital Marketing, Business and Innovation, Fintech, Blockchain, Agritech, Food Technology, Climate Tech, Environment and Circular Economy, Robotics and IoT, HealthTech and Biotech, Geospatial and Emerging Computing, Foundations and Test Preparation).	Dynamic taxonomy: primary categories, subcategories, tags, with stable ids and slugs. Admin create, edit and deactivate without a release; deactivation must not break old courses or links. One primary category per course plus many tags. Keep old slugs as aliases. Map the 74 free courses (today the import creates 10 categories). Support cross-category pathways such as AI for Agriculture.	High
Q2. Multi-provider content with attribution.	Provider registry as a reusable entity (creator, institution, company, research body, third-party platform, Gloxad). Attribution at lesson and resource level: creator, original title, platform, original URL, channel, avatar or neutral placeholder, ownership class (Gloxad-produced, licensed or partner, externally curated). Embed policy: official YouTube player or link only; never bypass disabled embedding; record permissions where needed. Today courses only carry a single videoCredit string.	High
Q3. 12-week programme as a 22-node trail.	Curriculum API (illustrative path: GET /api/lms/cohorts/:cohortId/curriculum) returning ordered nodes 1 to 22 with week, phase, title, description, activities, completion criteria and learner status. Nodes 1-10 are weeks 1-10 (five-day pattern: recorded, recorded, live, live, peer learning), nodes 11-16 are week 11 (assessment and practical work), nodes 17-22 are week 12 (reflection, final live class, comprehensive quizzes, final assessment, final project, overall review). Programme length and structure must be configurable. Keep lesson completion, assessment completion, task submission and node completion as separate states. Today the seeded plan has four weeks.	High

Question and decision	Backend work	Priority
Cross-cutting foundations	Stable ids and timestamps on categories, courses, providers, nodes, assessments, projects, submissions, recordings and workspaces; actor identity on state transitions; deactivation keeps history; server-validated workflow transitions; idempotent retries; audit history for grades, policies, approvals, deadlines and submission attempts.	High

Phase 2: academic governance

Question and decision	Backend work	Priority
Q4. Eight-stage final project: (1) topic selection, (2) proposal and objectives, (3) solution design and planning, (4) development milestone 1, (5) development milestone 2, (6) final submission, (7) presentation and defence, (8) final evaluation and sign-off.	Workflow state machine with stage records, submissions, tutor review (approve, reject, request revision), revision loops, notifications, and administrator final approval. The tutor publishes topics, sets deadlines and resubmission policy and recommends a mark; the administrator approves or returns it. Only after approval is the grade official. Separate instructor and administrator dashboards' data.	High
Q4. Reusable grading engine.	Assessment definitions (quizzes and tests, assignments, practical exercises, coding challenges, live attendance and participation, peer work, weekly tasks, final project, others), question types (auto and manual), rubrics, maximum marks, pass marks, deadlines, resubmission policy, attempts. Keep the raw mark separate from the weighted contribution (80/100 at 15% is 12/15). Validate that weights total 100%. Changes after an assessment starts need a recorded reason and administrator oversight.	High
Q4. Late and resubmission caps (general guideline).	Configurable score caps, example for a 15-mark project: on time 15, first deadline missed or first resubmission 10, second attempt 5, later 2, no submission or no resubmission after rejection 0. These are caps, not automatic grades. Record the policy that applied and show it to the student and administrator. Track attempts and penalties.	Medium
Q4. Gradebooks and audit.	Student gradebook (scores, outstanding work, how the overall result is calculated) and tutor class gradebook. Every manual grade change stores previous mark, new mark, reason, actor and time. Grading configuration changes are also audited. Quizzes auto-grade; assignments and projects are tutor graded.	High
Q5. Default grading and certificate rule.	Default weights when a tutor has not configured others: quizzes and tests 15%, assignments 15%, practical exercises 15%, coding challenges 10%, live attendance and participation 10%, weekly assessments and tasks 10%, final project 25% (total 100). Certificate passing threshold 70% overall, configurable by authorised programme-specific changes that are recorded. Eligibility also needs: all mandatory learning activities, required assessments, attendance where required, final project approved where applicable. Keep three states separate: learning completion, academic performance, certificate eligibility. A learner can finish the content and still be ineligible.	High

Phase 3: live learning

Question and decision	Backend work	Priority
Q6. LiveKit (WebRTC) as the primary classroom; YouTube Live stays secondary for webinars and large broadcasts.	Room creation and scheduling, access tokens and permissions from student and instructor dashboards, two-way audio and video, screen share, classroom controls, audio-only and quality adjustment, whiteboard needs a separate integration. Today the repo streams through YouTube (the 15 October change replaced LiveKit); both must be supported with LiveKit as default for regular teaching.	Medium

Question and decision	Backend work	Priority
Q6. Attendance.	Record join time, leave time and participation duration as attendance events. Recording views must never count as attendance; track them separately for grading and reporting.	Medium
Q6. Automatic recording.	Every scheduled LiveKit class records automatically when the instructor starts it and stops when it ends (evaluate LiveKit Egress or equivalent). Link each recording to course, trimester, week and class. Lifecycle states Processing, Available, Failed; ending a session does not mean the recording is ready. Secure storage, authenticated and time-limited playback, access for enrolled learners, the class instructor and administrators, retention and consent rules, tell participants when recording is active, future analytics.	Medium

Phase 4: persistent workspace

Question and decision	Backend work	Priority
Q7. One persistent workspace across lessons.	Workspace metadata and multiple files per learner; lesson-specific starter files; work survives navigation and leaving the platform; works for different languages and non-coding programmes; compatible with a future AI assistant without delaying core features.	Medium
Q7. Three-layer saving.	Layer 1 browser (IndexedDB) is frontend. Layer 2 is a backend cloud autosave endpoint (illustrative: PUT /api/lms/workspaces/:workspaceId/autosave) saving about 2 to 5 seconds after editing pauses, with version history and recovery, saving, saved, offline and failed states, safe retries and conflict handling without silent overwrites. Define payload limits, version semantics and the request and response contract. Layer 3 is GitHub (below).	Medium
Q7. Formal submission, both methods.	Direct Gloxad file submission and GitHub submission (repository URL plus commit id, private repositories via authorised access or an uploaded copy). Configurable per course or assessment. Each creates an immutable record: learner, course, assessment, exact version or commit, time. A later repository change must not change what the tutor assesses. Autosave, GitHub sync and formal submission never trigger each other silently.	Medium

Phase 5: developer workflow and verification

Question and decision	Backend work	Priority
Q7. Optional GitHub integration.	GitHub App or OAuth model with least privilege and secure token storage (learners never paste personal tokens). Connect and disconnect, create or link repositories, push and pull between workspace and GitHub, repository and commit information, error reporting. Optional for general access; selected advanced courses can require structured GitHub exercises.	Low
Q7. Server-side code execution.	Isolated runner (Judge0 or a properly isolated sandbox): several languages and runtime versions, time and memory limits, no network or system access, cannot reach databases, credentials, internal services or other learners' data, reliable test results fed into the grading engine. Client-side results are labelled Unverified and never become official grades; until the runner exists, tutors approve marks through the academic workflow. Needs an independent security review before running untrusted code.	Low

Roles (section 13)

Role	Backend must allow
Learner	View enrolled content, join classes, own workspace, save drafts, optionally connect GitHub, submit assessments, see own grades, feedback and recordings subject to access rules.

Role	Backend must allow
Tutor or instructor	Manage assigned classes, publish project topics, review student work, configure authorised assessment settings, grade and give feedback, manage classroom controls, access own class recordings, forward final project evaluations to the administrator.
Administrator	Manage taxonomy and platform settings, content and source records, programme policies, recording access, academic records, approve or return final project evaluations, oversee grading and certificate rules.

What the specification says engineering must deliver

- Before frontend integration is complete: detailed endpoint specifications, data schemas, error codes, authorisation rules and event contracts.
- Before implementation: compare the requirements with the existing code, list reusable services, conflicts and gaps, and record any gap, risk, workaround and target phase rather than leaving it out silently.
- Design items still open in the decisions register: taxonomy migration and admin interface; provider and resource schema with permission evidence; node and activity schema and completion model; workflow state machine, notifications and revision loops; grading formulas, rounding, missing-work treatment, grade scale and change governance; approval permissions and certificate issue controls; recording provider, storage, retention, consent and playback; workspace and file schema, autosave payload and conflict handling; GitHub App or OAuth model, permissions, webhooks and private repository access; sandbox provider, isolation, runtimes and limits.

Acceptance scenarios to automate (section 16.1)

- Create a category, map a course, deactivate the category, and confirm old references still work.
- Create a course with two providers and confirm attribution at lesson and resource level.
- Complete a lesson, an assessment and a submission separately and confirm the states stay distinct.
- Submit a project late and resubmit; confirm the configured cap is applied as a cap.
- Have a tutor recommend a grade; confirm the official result waits for the administrator.
- Run and record a class; confirm processing, replay access and separate attendance and replay records.
- Edit code offline, reconnect and sync; confirm no silent overwrite.
- Push to GitHub, change the repository, and confirm the submitted commit is still the one assessed.
- Submit client-side test results and confirm they stay Unverified until a trusted run or tutor approval.

D. Earlier gaps from the web app

These came from the Weekly Developer Guide and the web screens. Items that overlap the specification are marked.

Gap	Backend work	Priority
Assignments (also spec Phase 2)	Assignment, submission, rubric, grade and feedback models. Statuses: draft, submitted, late, under review, needs revision, graded, resubmitted. Accept text, code, PDF, ZIP, GitHub URL, website URL, video; validate type and size; never overwrite a submission or grade silently.	High
Join window on the server	The Join Live Class button opens 10 minutes before class; validate that window on the server, not only the browser.	High
Progress and final project screen data	Completion weights stored per course in an admin-editable table (the guide's placeholders: video 15, reading 10, sandbox 15, quiz 10, assignment 25, live attendance 15, weekly project 10). Reconcile these with the spec defaults before building.	High
Instructor tools	Cohort roster and progress, attendance, grading with rubric, feedback (approve or request revision), discussions (answer, pin, moderate), announcements, flags for learners who need support.	High
Admin tools	Programme setup (draft, publish, archive, preview), enrolment and instructor assignment, scheduling, scholarship review, completion and certificate issue controls, reports.	High

Gap	Backend work	Priority
Reports	Admin: enrolment counts, active learners, completion, weekly progress, assignment submission, attendance, final project status, certificates, scholarship statuses. Instructor: assigned cohorts and learners needing support.	Medium
Notifications (in app first)	Enrolment confirmation, new announcement, upcoming live class, assignment due date, submission received, grading and feedback, scholarship update, certificate issued.	Medium
Paid trimester enrolment	A paid trimester currently returns 402. Connect checkout (Paystack is already integrated for courses) and decide the enrolment policy: free or paid, prerequisites, closing date, withdrawal.	Medium
Discussions	Discussion, post and reply models; students ask, reply and share; instructors answer, pin, moderate.	Medium
Program certificates	A certificate links one learner to one completed programme; statuses not eligible, eligible, issued, revoked (revocation needs an audit reason). Apply the spec's 70% rule and completion conditions.	Medium
Scholarship documents	Application documents are stored as links only today; add upload using the existing presigned S3 flow and validate types and sizes.	Low
Course content fields	Transcript and course materials per lesson (the player has tabs for them but no data).	Low
Learner extras	AI Tutor, study streak, weekly goal and discussions data for the home and My Courses screens (later phase; never a blocker).	Low
Design task upload	An endpoint for the design-task submission shown in the Figma course screens.	Low
Course filter	Optional ?source=tutor filter on GET /courses so the web app can tell tutor courses from catalogue courses without comparing titles.	Low

Suggested order

Step	Work	Why first
1	Release part A and the production steps in part B	Makes the interactive free courses live; the code and tests already exist
2	Taxonomy, provider attribution, curriculum nodes, audit history (Phase 1)	Everything else builds on these records
3	Assignments, grading engine, gradebooks, 8-stage project, certificate rules (Phase 2)	Unlocks the instructor and admin screens and certificates
4	Instructor and admin endpoints, reports, notifications (part D)	Lets staff run a cohort
5	LiveKit, recording, workspace, GitHub and code execution (Phases 3 to 5)	Largest and riskiest; needs a security review for code execution